Developer Handoff Brief — GamerGain / PlayEarning Nexus

Scope: launch on web + native apps (Android & iOS)

You're being brought in to **stand up, deploy, and ship** an app whose code is already written. This is a configuration/integration/deployment job, not a build-from-scratch job. Read this page, then the linked docs, and you'll have everything you need. Working from the pre-built execution kit, estimated total: **~30–45 hours at $75/hour = ~$2,250–$3,375** for the full PWA + Android + iOS launch, **plus ~8–12 h (~$600–900) for the load test** that measures how many users the app can handle — a grand total of **~$2,850–$4,275** (the kit's added automation drives this down). (External waits — app-store review, lawyer sign-off — don't consume
your time, and the owner-provided accounts/fees are separate.)

1. Architecture in 30 seconds

- **Frontend:** React + Vite **PWA** (208 pages). All server calls go through one module, `src/api/base44Client.js`, over HTTP. Configured with `VITE_NEXUS_API_URL`.
- **Backend:** self-hosted **Deno** service in `/backend` (formerly Base44). It mounts **526 functions** as HTTP routes, has **239 Postgres tables**, JWT + Google auth, an agent runtime, and a cron scheduler. Runs from `backend/server/main.ts`; Docker + docker-compose included.
- **Database: PostgreSQL** (schema generated in `backend/db/schema.sql`, validated on PG 16).
- **Native: Capacitor** wrapper (wrapper-only — `android/` / `ios/` are regenerated with `npm run native:regenerate`, not committed).
- **Integrations to wire (keys provided by owner):** Stripe, PayPal, Twilio (SMS), BitLabs (surveys), OpenAI or Anthropic (LLM), SendGrid or SES (email), Google OAuth, S3 (uploads).

2. What's already done — do NOT rebuild

Full frontend + backend; auth (signup/login/password-reset/Google); all 526 functions converted to the self-hosted SDK; the 239-table schema; row-level security; agent runtime; scheduler; migration tooling; brand-matched auth screens; and the GamerGain icon set. It **compiles and passes syntax checks, and the DB layer is validated against real Postgres** — but it has **not been booted end-to-end as a live service yet.** That first boot is your Phase A.

3. Your work, in order (with rough effort)

Phase	Task	Effort
A. Run it locally	`cd backend && cp .env.example .env`, `docker compose up`, load `db/schema.sql` + `db/seed.sql`, run `deno run … tools/smoke-test.ts`. Fix any query-translator edge cases in `sdk/db.ts` that surface (`$or`, array-contains, pagination). **This is the de-risking step — do it first.**	0.5–1 day
B. Wire integrations	Paste the owner's API keys into backend `.env` and frontend `.env.local`. Verify a test-mode Stripe/PayPal charge, an LLM call, an email send.	0.5–1 day
C. Deploy	Backend (Deno) + managed Postgres + frontend static build. **Recommended for speed:** a container host (Render / Railway / Fly.io / AWS App Runner) + managed PG (Neon / Supabase / RDS) + Amplify/CloudFront for the frontend. Set the SPA fallback (`404/403 → index.html`) and HTTPS + custom domain.	1–2 days

D. User accounts / auth	Set `AUTH_JWT_SECRET`; wire email provider for password reset; set up Google OAuth (`GOOGLE_CLIENT_ID` both sides) if wanted; verify signup→login→reset end-to-end (Mailhog steps in the Phase-2 runbook).	0.5 day
E. Native apps	`npm run native:regenerate`; Android → signed `.aab` in Android Studio → Play Console; iOS → archive in Xcode (**Mac required**) → App Store Connect. Follow `APP-STORE-SUBMISSION-CHECKLIST.md`.	2–4 days
F. QA + go-live	End-to-end pass on the go-live checklist (`MASTER-LAUNCH-GUIDE.md` Phase 10); turn on the backend cron schedules.	1 day

4. What the owner provides (so you're never blocked)

- **Filled-in API-key sheet** (from `CONFIG-AND-SECRETS.md`) — all values, ready to paste.
- **Domain**, a **hosting account**, an **Apple Developer** account ($99/yr) and **Google Play Console** account ($25). iOS work needs a **Mac with Xcode**.
- **Legal pages** completed + lawyer-reviewed (templates + `LEGAL-PAGES-GUIDE.md`).
- The **app icon** is already in the repo (GamerGain green "G", `assets/icon.png`).

5. Where to spend care (known gotchas)

- **Deno, not Node** — the backend runs on Deno; deploy with a Deno-capable container (Dockerfile provided), not a Node buildpack.
- **Query translator** (`backend/sdk/db.ts`): equality/operators/JSONB are done and tested; add `$or`/nested-boolean/array-contains only if a function needs them (flush out in Phase A).
- **Agents** need `OPENAI_API_KEY` to generate replies; **UploadFile** needs `S3_BUCKET` + AWS creds.
- **Earn-money app = extra app-store scrutiny.** Present it as a real app, keep the prize pool framed as skill/merit-based (not gambling), and give reviewers a demo login. See the checklist.
- `functions.invoke` returns Base44-style `{ data }` on purpose — don't "simplify" it; 58 frontend files depend on that shape.

6. Read these first (in the repo root and `/backend`)

`DE-BASE44-REWORK.md` (what the stack is) → `backend/PHASE-2-RUNBOOK.md` (get it running) → `MASTER-LAUNCH-GUIDE.md` (full sequence) → `CONFIG-AND-SECRETS.md` (keys) → `MOBILE-APP-WRAPPER-GUIDE.md` + `APP-STORE-SUBMISSION-CHECKLIST.md` (native) → `BASE44-TO-SELFHOSTED-MAP.md` (how the API surface maps, if you're curious about the migration).

7. Definition of done

- [ ] Backend live (managed PG + Deno host), `/health` green, scheduler on
- [ ] Frontend live on the domain with HTTPS; deep links work
- [ ] Signup / login / password-reset / (Google) all working against production
- [ ] Test-mode Stripe + PayPal transaction succeeds; a payout path tested
- [ ] A survey → reward credit and a function invocation both work end-to-end
- [ ] Privacy/Terms live at public URLs
- [ ] Android `.aab` submitted to Play; iOS archive submitted to App Store

8. How to keep the bill down (owner notes)

- Owner creates all the third-party **accounts + keys** — don't pay dev hours for signup forms.
- Have the dev do **Phase A (local boot) before anything else** — it's cheap and de-risks the rest.
- Use **managed Postgres + a container host** rather than hand-built AWS to save DevOps days.
- **Legal review runs in parallel** — it's not developer work and shouldn't block them.